IT404041 REAL TIME PROGRAMMING · ASSIGNMENT 2 · SMART GREENHOUSE CONTROLLER

CLI Explained

Your screen, line by line – words, processes and scenarios in simple language

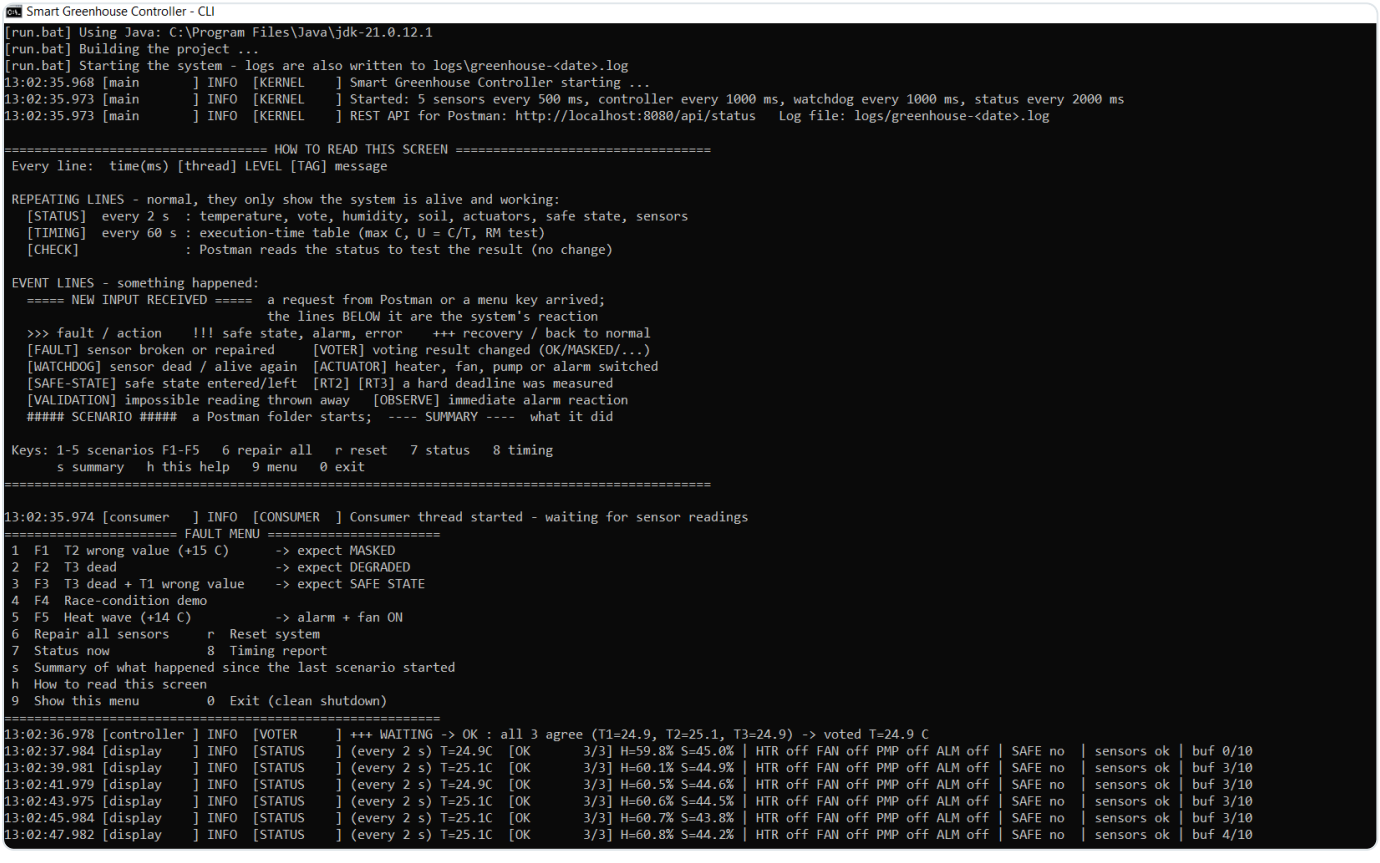
Generated 2026-11-05

This document explains the black CLI window in **simple words**, using **real screenshots** of the running system.

Part	What it covers
Part 1	What happens when the system starts (screenshot 1)
Part 2	Why the same lines repeat when nobody sends anything (screenshots 2 and 3)
Part 3	How to read the timing table
Part 4	What happens when Postman starts a scenario (screenshot 4)
Part 5	What you will see for **every** Postman scenario
Part 6	Dictionary of all the technical words, with examples

> **The most important idea:** the system is like a **heart that keeps beating**. It works all the time, even when nobody touches it. Most lines on the screen are just this heartbeat. Only the lines **under a** `NEW INPUT RECEIVED` **box** are reactions to something you did.

Part 1 – What happens at the start (screenshot 1)



Read it from the top to the bottom:

Step 1 – `run.bat` prepares everything

```
[run.bat] Using Java: C:\Program Files\Java\jdk-21.0.12.1
[run.bat] Building the project ...
```

Line	Meaning
`Using Java: ...jdk-21...`	The start script found Java 21 on your PC
`Building the project ...`	It compiles all the Java code into one program file (`smart-greenhouse.jar`)

Then the window is **cleared**, and the new session starts with this box:

```
================================================================================
  NEW SESSION  started 2026-11-05 16:38:39
  Full log   : logs/2026-11-05/greenhouse_16-38-39.log   (exact copy of this window)
  Events log : logs/2026-11-05/events_16-38-39.log   (only inputs, events and summaries)
================================================================================
```

Line	Meaning
`NEW SESSION started ...`	When this run began. Every start of the system is a new session.
`Full log`	**Everything you see in this window from here on is written into this file, exactly the same.**
`Events log`	A shorter file with only the inputs, the important events and the summaries

(Your screenshot 1 below is from an older version, before the window was cleared and the session box was added.)

Step 2 – the kernel starts all the jobs

```
13:02:35.968 [main   ] INFO [KERNEL] Smart Greenhouse Controller starting ...
13:02:35.973 [main   ] INFO [KERNEL] Started: 5 sensors every 500 ms, controller every 1000 ms,
                                     watchdog every 1000 ms, status every 2000 ms
13:02:35.973 [main   ] INFO [KERNEL] REST API for Postman: http://localhost:8080/api/status
```

The **kernel** is the "boss" of the program. It starts these jobs, and each one repeats forever:

Job	Repeats every	Its job in one sentence
5 **sensors** (T1, T2, T3 temperature · H1 humidity · S1 soil)	500 ms = 0.5 s	Measure the greenhouse
Controller	1000 ms = 1 s	Decide: heater / fan / pump / alarm ON or OFF
Watchdog	1000 ms = 1 s	Check that every sensor is still sending
Status display	2000 ms = 2 s	Print one summary line

The last line gives the address that Postman uses to talk to the system.

Step 3 – the "HOW TO READ THIS SCREEN" box

A small help text that is printed once. Press `h` in the window to show it again.

Step 4 – the consumer thread starts

```
13:02:35.974 [consumer] INFO [CONSUMER] Consumer thread started - waiting for sensor readings
```

The **consumer** is a worker that takes every sensor reading out of the waiting box (the **buffer**), checks it and gives it to the controller. It is now waiting for the first readings.

Step 5 – the FAULT MENU

The list of keys you can type into this window: `1`–`5` start the scenarios F1–F5 without Postman, `6` repairs all sensors, `0` stops the system.

Step 6 – the very first vote

```
13:02:36.978 [controller] INFO [VOTER] +++ WAITING -> OK : all 3 agree (T1=24.9, T2=25.1, T3=24.9) -> voted T=24.9 C
```

This is the controller's **first decision**, 1 second after the start:

Piece	Meaning
`+++`	Good news / back to normal
`WAITING -> OK`	Before: no vote yet (WAITING). Now: the vote is OK.
`all 3 agree`	The 3 temperature sensors show almost the same value (difference ≤ 1 °C)
`T1=24.9, T2=25.1, T3=24.9`	What each temperature sensor measured
`voted T=24.9 C`	The temperature the system **believes** and uses for its decisions (the middle value of the three)

Step 7 – the status line (repeats every 2 s)

```
13:02:37.984 [display] INFO [STATUS] (every 2 s) T=24.9C [OK 3/3] H=59.8% S=45.0% | Heater off Fan off Pump off Alarm off | SAFE no | sensors ok | buf 0/10
```

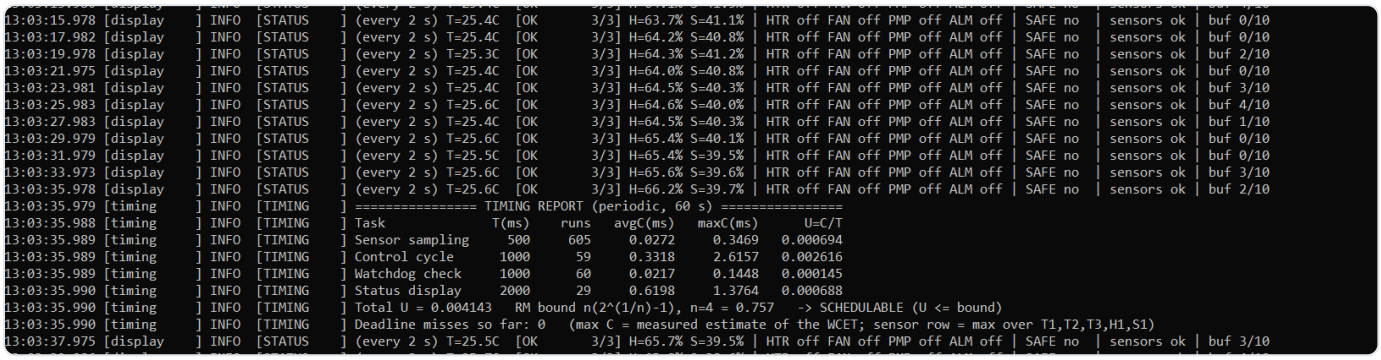
Every part of it:

Part	Meaning	Example
`(every 2 s)`	This line repeats every 2 seconds. It is **not** an event.	–
`T=24.9C`	Voted temperature	24.9 °C
`[OK 3/3]`	Vote result, and how many of the 3 temperature sensors were used	`[MASKED 2/3]` = one sensor was ignored
`H=59.8%`	Humidity (water in the air)	above 85 % → fan ON
`S=45.0%`	Soil moisture (water in the soil)	below 35 % → pump ON
`Heater off`	Heater	`Heater ON` when T < 22 °C
`Fan off`	Fan	`Fan ON` when T > 28 °C or H > 85 %
`Pump off`	Pump	`Pump ON` when soil < 35 %
`Alarm off`	Alarm	`Alarm ON` when T > 35 °C or in the safe state
`SAFE no`	Is the system in the **safe state**?	`SAFE YES` = emergency mode
`sensors ok`	All 5 sensors are fine	`T3:FAILED` = T3 is dead

Part	Meaning	Example
`buf 0/10`	How many readings are waiting in the buffer right now (of 10 places)	`buf 3/10` is normal

Capital letters = ON. `Fan ON` is easy to spot among the `off` words.

Part 2 – Nobody sends anything, but lines keep coming (screenshots 2 and 3)



What you see: only `[STATUS]` lines, every 2 seconds, plus a `[TIMING]` table every 60 seconds.

What it means: the system is **alive and watching**, and nothing is wrong. Between two status lines (2 seconds), inside the system:

Job	Every 1 s	Between two status lines (2 s)
Sensor readings (5 sensors, each every 0.5 s)	10	**20**
Controller votes	1	**2**
Watchdog checks	1	**2**
Status lines printed	–	**1**

They print nothing because **nothing changed**. The system only writes a line when something changes.

Why do the numbers move slowly by themselves?

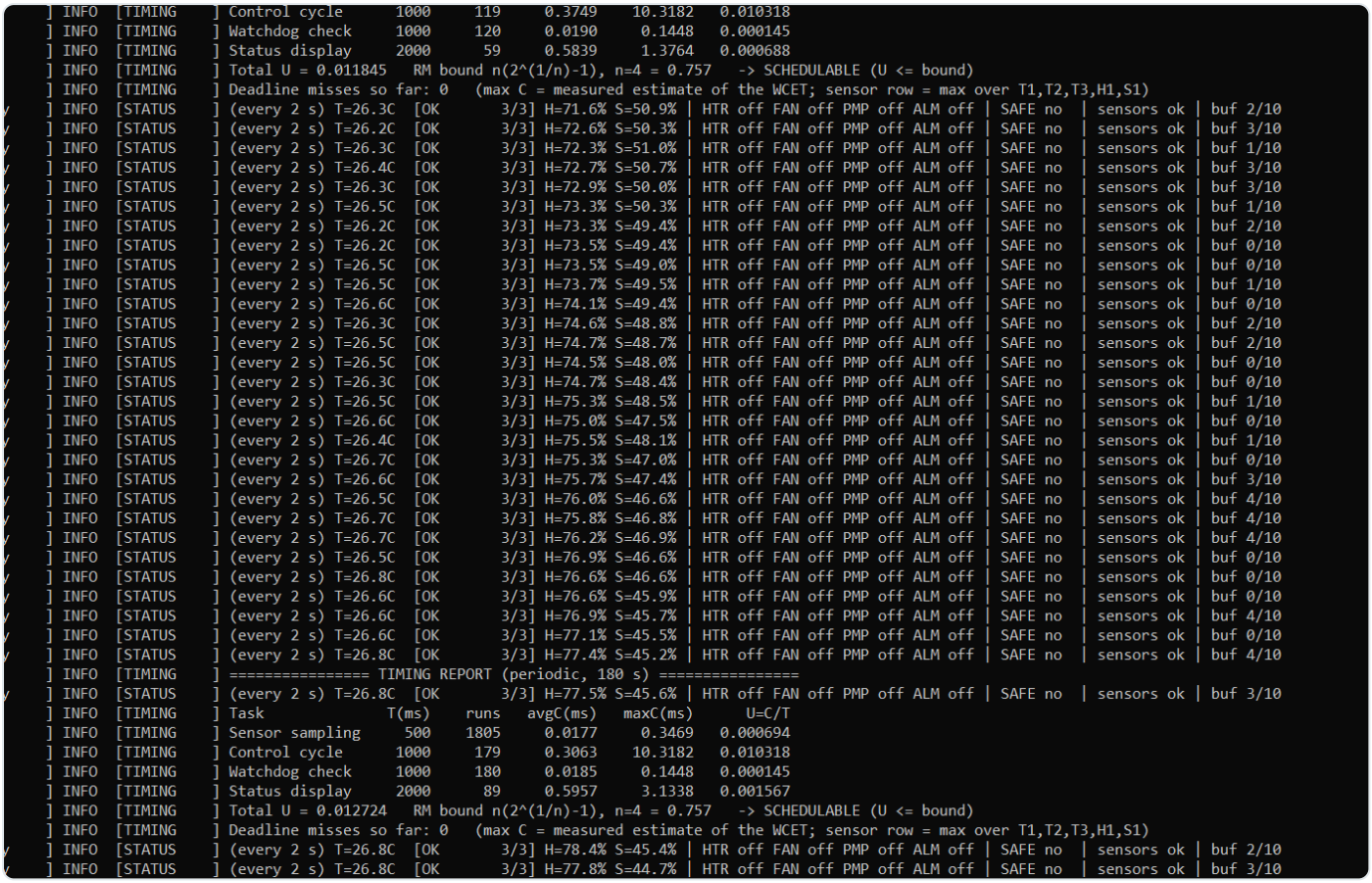
Look at screenshot 2: `T=25.4C → 25.6C`, `H=63.7% → 66.2%`, `S=41.1% → 39.7%`. Nobody sent anything, so why?

Because the greenhouse is **simulated like a real one**. Every second:

Nature in the simulation	Effect per second	You see
The sun warms the air	+0.01 °C	T slowly goes up
Plants breathe out water	+0.1 % humidity	H slowly goes up
The soil dries	−0.1 % soil	S slowly goes down
Heater ON / fan ON / pump ON	+0.5 °C / −0.5 °C and −1 % H / +2 % soil	(only when switched on)

Small jumps like 25.4 → 25.3 → 25.4 are **sensor noise**: real sensors never measure exactly the same twice.

The system reacts by itself when a limit is reached



In screenshot 3 the humidity has risen to **77.8 %** and the soil has dried to **44.7 %**. If you leave the system running:

- **After about 70 more seconds** H passes 85 %, and you will see `ACTUATOR FAN -> ON (T=26.9 C, H=85.1 %)`. The fan dries the air, and when H is back under 85 % you see `FAN -> OFF`.
- **When the soil falls below 35 %** you will see `ACTUATOR PUMP -> ON (soil=34.9 % < 35.0 %)`. You saw exactly this line in an earlier run. The pump stays on until the soil is above 50 %.

This is **Environmental Control**: the controller keeps the greenhouse inside its limits on its own, without any Postman input.

Part 3 – The timing table (every 60 s)

From screenshot 2 (after 60 s):

```
============== TIMING REPORT (periodic, 60 s) ==============
Task              T(ms)   runs   avgC(ms)   maxC(ms)     U=C/T
Sensor sampling     500    605     0.0272     0.3469   0.000694
Control cycle      1000     59     0.3318     2.6157   0.002616
Watchdog check     1000     60     0.0217     0.1448   0.000145
Status display     2000     29     0.6198     1.3764   0.000688
Total U = 0.004143   RM bound n(2^(1/n)-1), n=4 = 0.757   -> SCHEDULABLE (U <= bound)
Deadline misses so far: 0
```

Column	Simple meaning	Example (Sensor sampling row)
Task	Which repeating job	reading the sensors
T(ms)	Its **period**: how often it must run	every 500 ms
runs	How many times it has run so far	605 (5 sensors × 2 per second × 60 s ≈ 600)
avgC(ms)	**Average** time one run takes	0.0272 ms
maxC(ms)	**Longest** time one run took. This is the worst case, and it is the important one.	0.3469 ms
U=C/T	**Utilisation**: what part of the CPU this job needs (max C ÷ T)	0.3469 ÷ 500 = 0.000694 = 0.07 %

The last two lines:

Line	Meaning
`Total U = 0.004143`	All 4 jobs together need about **0.4 %** of the CPU in the worst case
`RM bound ... = 0.757`	The **Rate-Monotonic** rule: if total U is at most 0.757 (75.7 %), all 4 jobs always finish before their next start (n = 4 jobs)
`SCHEDULABLE (U <= bound)`	0.004 ≤ 0.757, so the task set is OK
`Deadline misses so far: 0`	No job was late even once

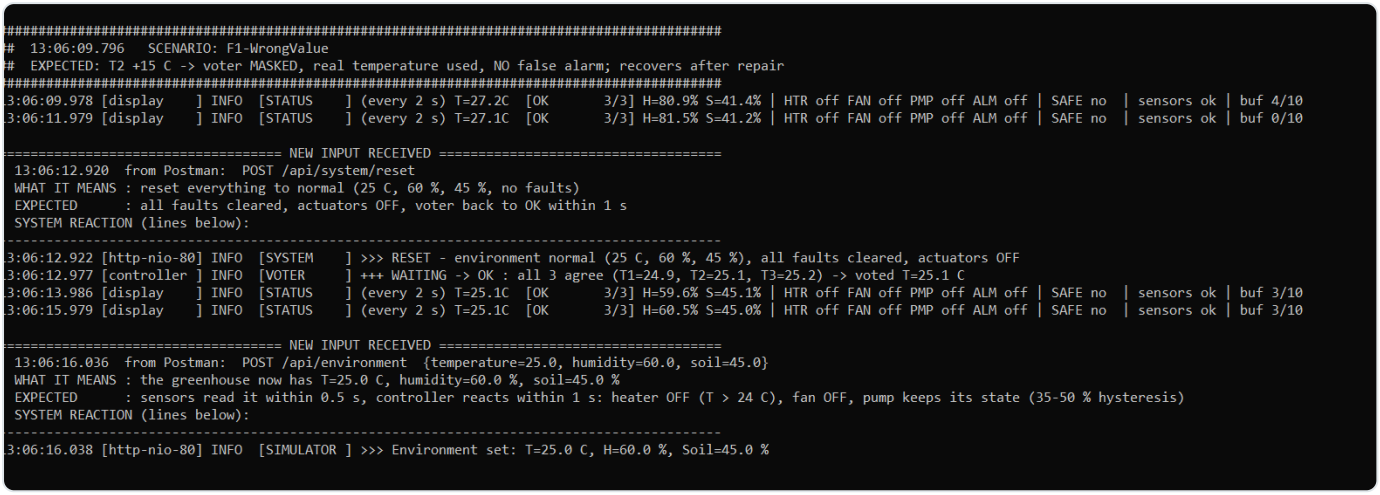
Why did "Control cycle maxC" jump to 10.3 ms in screenshot 3?

At 60 s it was 2.6 ms; at 120 s and 180 s it is **10.3 ms**. Once, one control cycle was slower. The usual causes:

- **Garbage collection:** Java paused for a moment to clean up memory.
- **Windows** gave the CPU to another program.
- The **JIT** compiler was optimising code.

This is exactly why the table keeps the **maximum**, and why the report says *normal Java cannot guarantee deadlines*. 10.3 ms is still far below the 1000 ms period, so U is still tiny (0.0127) and the set is still schedulable.

Part 4 – Postman starts a scenario (screenshot 4)



The `#####` box – a scenario starts

```
##############################################################################################################
##  SCENARIO: F1-WrongValue   (start of a new test - the inputs of this scenario follow below)
##  16:48:55.570  from Postman:  POST /api/scenario/start  {name=F1-WrongValue}
##  EXPECTED: T2 +15 C -> voter MASKED, real temperature used, NO false alarm; recovers after repair
##############################################################################################################
```

This box is the answer to the **first request of every Postman folder**, `▶ Scenario start` (`POST /api/scenario/start`):

1. **SCENARIO:** the folder that begins, here **F1-WrongValue**.
2. **from Postman:** the request that started it.
3. **EXPECTED:** what **should** happen.

The `NEW INPUT RECEIVED` boxes of this scenario follow below it.

The **last request** of every folder, `■ Scenario summary` (`POST /api/scenario/end`), prints the `SUMMARY` box. It lists what **really** happened:

```
  SUMMARY: F1-WrongValue   (6.8 s, 11 events) - everything important that happened in this scenario:
  16:49:02.346  from Postman:  POST /api/scenario/end
```

(Screenshot 4 above is from the older version, where the box did not yet show which request caused it.)

The two `[STATUS]` lines after it are the heartbeat again: the system keeps running while Postman works.

The first input – reset

```
================ NEW INPUT RECEIVED ================
  13:06:12.920  from Postman:  POST /api/system/reset
  WHAT IT MEANS : reset everything to normal (25 C, 60 %, 45 %, no faults)
  EXPECTED      : all faults cleared, actuators OFF, voter back to OK within 1 s
  SYSTEM REACTION (lines below):
-----------------------------------------------------
13:06:12.922 [http-nio-80] INFO [SYSTEM] >>> RESET - environment normal (25 C, 60 %, 45 %), all faults
cleared, actuators OFF
13:06:12.977 [controller ] INFO [VOTER ] +++ WAITING -> OK : all 3 agree (T1=24.9, T2=25.1, T3=25.2) -> voted
T=25.1 C
```

Line	What happened
Box	Postman sent **reset**: start clean, so old faults can't disturb this test
`[http-nio-80] ... RESET`	The thread that received the Postman request did the reset immediately (2 ms later)
`WAITING -> OK`	After a reset the voter starts fresh (WAITING = no vote yet). At the next control cycle (55 ms later) the 3 sensors agree, so OK.
Next `[STATUS]`	`T=25.1C` – the temperature jumped from 27.2 back to 25 because the reset set the greenhouse to normal

The second input – set the environment

```
  13:06:16.036  from Postman:  POST /api/environment  {temperature=25.0, humidity=60.0, soil=45.0}
  WHAT IT MEANS : the greenhouse now has T=25.0 C, humidity=60.0 %, soil=45.0 %
  EXPECTED      : sensors read it within 0.5 s, controller reacts within 1 s: heater OFF (T > 24 C), fan OFF,
pump keeps its state ...
13:06:16.038 [http-nio-80] INFO [SIMULATOR] >>> Environment set: T=25.0 C, H=60.0 %, Soil=45.0 %
```

The simulated greenhouse now has exactly these values. **Why is there no `[ACTUATOR]` line?** Heater, fan and pump were already off, and these values keep them off. **Nothing changed, so nothing is printed.**

Next, Postman reads the status (a `[CHECK]` line) to confirm "vote OK", and then injects the fault into T2. Part 5 continues from here.

Part 5 – What you will see for every Postman scenario

Every scenario follows the same pattern:

```
##### SCENARIO box #####          1st request: POST /api/scenario/start (Postman starts the folder)
  NEW INPUT: reset                clean start
  NEW INPUT: environment          normal values
  [CHECK]                         Postman confirms "OK"
  NEW INPUT: the fault / action   the real test
     ... reaction lines ...        ← THIS is what the scenario shows
  [CHECK]                         Postman confirms the reaction (test passes / fails)
  NEW INPUT: repair               fix the fault
     ... recovery lines ...        +++ back to normal
  [CHECK]                         Postman confirms the recovery
----- SUMMARY -----               last request: POST /api/scenario/end (numbered list of everything important)
```

00-Common-Setup – is the system OK?

Postman sends	CLI shows	Meaning
GET status	`[CHECK] Postman reads the status`	Is the system running?
reset + environment 25 °C	`RESET`, `Environment set`	Clean, normal start
GET status (after 2.5 s)	`[CHECK] ... vote OK, safe state no`	Everything is normal

01-Normal-Operation – the control rules, no faults

Postman sends	CLI reaction	Rule shown
environment T=20 °C	`ACTUATOR HEATER -> ON (T=20.1 C < 22.0 C)`	heater ON below 22 °C
T=30 °C	`HEATER -> OFF (T > 24 C)`, `FAN -> ON (T=30.0 C ...)`	heater OFF above 24, fan ON above 28
H=90 %	`FAN -> ON (... H=90.0 %)`	fan ON above 85 % humidity
soil=30 %	`PUMP -> ON (soil=30.x % < 35.0 %)`	pump ON below 35 %
soil=55 %	`PUMP -> OFF (soil=55.x % > 50.0 %)`	pump OFF above 50 %
T=21, then T=22.5	heater ON, then **no** heater line	**hysteresis**: between 22 and 24 °C the heater keeps its state

F1-WrongValue – one temperature sensor lies

Postman sends	CLI reaction	Meaning
fault T2 WRONG_VALUE +15	`FAULT injected: T2 -> WRONG_VALUE (+15.0 C)`	T2 now reports ~40 °C instead of 25 °C
	`VOTER OK -> MASKED : T2=39.9 C out-voted, T1 and T3 agree -> voted T=25.0 C`	2 sensors say 25, 1 says 40, so the majority wins and T2 is ignored
	`[STATUS] ... [MASKED 2/3] ... Alarm off ... T2:WRONG_VALUE`	**No false alarm**: the system still uses 25 °C
repair T2	`+++ T2 repaired`, `+++ MASKED -> OK : all 3 agree`	back to normal

Proves: one broken sensor cannot make the greenhouse do something wrong (**TMR masking**).

F2-DeadSensor – one temperature sensor stops sending

Postman sends	CLI reaction	Meaning
fault T3 DEAD	`FAULT injected: T3 -> DEAD`	T3 sends nothing any more
(wait ~2 s)	`VOTER OK -> DEGRADED : only 2 sensors (T1, T2) - they agree`	the system continues with 2 sensors
	`WATCHDOG !!! T3 FAILED - no heartbeat for 2001 ms (timeout 1500 ms)`	no reading from T3 for more than 1.5 s, so T3 is declared dead
	`[RT3 HARD] T3 detected dead 1611 ms after it stopped -> MET (deadline 2500 ms)`	detected in 1.6 s; the rule says at most 2.5 s, so **met**
repair T3	`+++ T3 RECOVERED - heartbeats are arriving again`, `DEGRADED -> OK`	T3 is back

Proves: a dead sensor is detected in time, and the system keeps working with less (**graceful degradation**).

F3-SafeState – two sensors bad, the system cannot trust anything

Postman sends	CLI reaction	Meaning
fault T3 DEAD	as in F2: `T3 FAILED`, `DEGRADED`	2 sensors left
fault T1 WRONG_VALUE +15	`DEGRADED -> FAILED : only 2 sensors and they DISAGREE (T1=40.0, T2=24.9)`	40 or 25? Nobody can know which one is right.
	`!!! ENTER SAFE STATE`, `FAN -> ON (safe state)`, `ALARM -> ON (safe state)`	**emergency mode:** heater OFF, fan ON, alarm ON
(wait)	`!!! SAFE STATE ACTIVE for 1 s ...` (every second)	warns again and again while the problem lasts
repair T1 and T3	`Good vote 1/2`, then `+++ EXIT SAFE STATE after 2 consecutive good votes`	leaves the safe state only after 2 good votes in a row
	`ALARM -> OFF`, `FAN -> OFF`	normal control again

Proves: when the data cannot be trusted, the system goes to a **safe state** instead of guessing, and comes back by itself.

F4-RaceCondition – two threads change one number

Postman sends	CLI reaction	Meaning
POST demo/race	`Unsafe counter = 130173 (expected 200000, LOST 69827 updates)`	2 threads added 100 000 each **without protection** and overwrote each other's work
	`+++ Synchronized counter = 200000 CORRECT`	fix 1: only one thread at a time
	`+++ AtomicInteger = 200000 CORRECT`	fix 2: an "atomic" (unbreakable) add

Proves: shared data must be protected. The unsafe number is different on every run.

F5-HeatWave – sudden high temperature

Postman sends	CLI reaction	Meaning
heat wave +14 °C	`SIMULATOR HEAT WAVE +14.0 C -> real temperature now 39.0 C`	the greenhouse suddenly has 39 °C
	`OBSERVE Observe & React: voted T=39.0 C > 35 C -> immediate reaction`	the system does **not** wait for the next 1 s control cycle
	`ALARM -> ON`, `FAN -> ON (over-temperature)`	alarm and cooling
	`[RT2 HARD] alarm ON 37 ms after temperature went above 35 C -> MET (deadline 1000 ms)`	reacted in 0.037 s; the rule says within 1 s
reset	`ALARM -> OFF`	back to normal

Proves: a dangerous situation gets a reaction within its hard deadline.

X1-InvalidInputs – wrong data from outside

Every wrong request gets a box like this:

```
================ NEW INPUT RECEIVED ================
  from Postman:  POST /api/environment
  RESULT       : REJECTED - 400 Validation failed [temperature must be <= 80 C (got 500.0)]
  SYSTEM REACTION: none - invalid input is refused, the greenhouse keeps running unchanged
```

The 12 tests: temperature 500, humidity −5, `"hot"` instead of a number, empty data, broken JSON, unknown sensor `T9`, unknown fault `EXPLODE`, missing value, value 500, heat wave 1000, an unknown address, and a wrong HTTP method. At the end the `[CHECK]` still shows **vote OK**.

Proves: bad input can never crash or change the system.

X2-StuckAndImpossibleValue

Postman sends	CLI reaction	Meaning
T1 STUCK, then T=30 °C	`FAULT ... T1 -> STUCK at 25.1 C`, `OK -> MASKED : T1=25.1 C out-voted`	T1 still says 25 while the real value is 30, so T1 is ignored
T2 WRONG_VALUE +90	`VALIDATION Reading of T2 = 115.0 C REJECTED (outside physical range -40..80 C)`	impossible value thrown away
	`-> DEGRADED : only 2 sensors`	the system continues with T1 and T3

X3-HumiditySoilFailure – sensors without a backup

Postman sends	CLI reaction	Meaning
soil 30 %	`PUMP -> ON`	normal
S1 DEAD	`WATCHDOG S1 FAILED`, `PUMP -> OFF (soil sensor failed - fail-safe)`, `WARNING soil sensor S1 not trustworthy` every second	without a soil value, OFF is safer (no flooding)
H1 DEAD	`WATCHDOG H1 FAILED`, `FAN -> ON (humidity sensor H1 not trustworthy - fan ON as precaution)`	without a humidity value, the fan runs to be safe
repair both	`+++ H1 RECOVERED`, `+++ S1 RECOVERED`, `FAN -> OFF`, `PUMP -> ON`	normal control again

X4-MixedChaos – everything at once

Valid inputs, invalid inputs (REJECTED boxes), T2 wrong, H1 dead, a **software exception**, and a heat wave, all mixed together. Watch for these lines:

```
!!! Exception in control cycle: simulated software fault -> forward recovery to SAFE STATE
!!! ENTER SAFE STATE - exception in control cycle ...
+++ EXIT SAFE STATE after 2 consecutive good votes
>>> ALARM -> ON (T=38.x C > 35.0 C)
```

Proves: even a program error does not crash the system. It goes to the safe state, keeps running, and recovers.

02-Repair-All and 03-Timing-And-Shutdown

- **02** repairs every sensor (`+++ ... repaired` lines) and resets.
- **03** prints the timing table and then stops the system:

  ```
  Shutting down ...
  SAFE-STATE  Shutdown - actuators set to safe position
  CONSUMER    Consumer thread stopped (interrupted) - clean shutdown
  TIMING      ======== TIMING REPORT (FINAL, ... s) ========
  KERNEL      Clean shutdown complete - all real-time threads stopped
  ```

Part 6 – Dictionary of the technical words

Words in every line

Word	Simple meaning	Example from the screen
timestamp	exact time, down to the millisecond (1/1000 s)	`13:06:12.920` = 13 h 06 min 12 s 920 ms
thread	a worker inside the program; many work at the same time	`[controller]`, `[display]`, `[consumer]`
http-nio-80…	the worker that receives Postman requests (port 8080, the name is cut)	`[http-nio-80] ... RESET`
main	the worker that starts the program	`[main] ... starting`
INFO / WARN / ERROR / CRIT	normal / not normal / fault found / a human is needed	`ERROR [WATCHDOG] T3 FAILED`
tag	which part of the system wrote the line	`[VOTER]`, `[ACTUATOR]`
`>>>` `!!!` `+++`	action or fault / danger / good news	`+++ EXIT SAFE STATE`

The greenhouse

Word	Simple meaning
simulator	a program that pretends to be the real greenhouse (we have no real hardware)
sensor	measures something: T1, T2, T3 = temperature, H1 = humidity, S1 = soil moisture
actuator	something that acts: heater, fan, pump, alarm
humidity (H)	how much water is in the air, in %
soil moisture (S)	how wet the soil is, in %
heat wave	a sudden jump in temperature

Real-time words

Word	Simple meaning	Example
real-time system	a system where the answer must be correct **and on time**	the alarm must come within 1 s
periodic task	a job that repeats at a fixed rhythm	sensors every 500 ms
period (T)	the time between two runs	T = 1000 ms for the controller
deadline	the latest time a reaction may happen	alarm within 1000 ms
hard deadline	missing it is dangerous	RT2 alarm, RT3 watchdog
firm deadline	a late result is useless, but not dangerous	RT1 sensor reading, RT4 control cycle
soft deadline	late is a bit worse, but still OK	RT5 status line
RT1–RT5	the 5 timing rules of the assignment	`[RT3 HARD] ... MET`
MET / MISSED	the deadline was kept / not kept	`MET (deadline 2500 ms)`
execution time (C)	how long one run of a job takes	maxC = 0.35 ms

Word	Simple meaning	Example
WCET	Worst-Case Execution Time: the longest a job can take. We **estimate** it with max C.	–
utilisation (U)	the part of the CPU a job needs, U = C ÷ T	0.000694 = 0.07 %
RM bound / Rate-Monotonic	rule: if total U ≤ 0.757 (for 4 jobs), all jobs are always on time	`SCHEDULABLE`
schedulable	all jobs can always finish before their deadlines	–
deadline miss	a job started late or ran too long	`Deadline misses so far: 0`
garbage collection (GC)	Java pausing to clean unused memory; it can make a job late	the 10.3 ms max C
JIT	Java optimising code while running; the first runs are slower	–

Fault-tolerance words

Word	Simple meaning	Example
fault	the **cause** of a problem	sensor T2 is broken
error	the **wrong data** caused by it	T2 reports 40 °C
failure	a **wrong action** that the outside world sees	heater ON at 40 °C – our system prevents this
fault injection	breaking a sensor on purpose, to test	`FAULT injected: T2 -> WRONG_VALUE`
WRONG_VALUE / STUCK / DEAD	adds an offset / freezes / sends nothing	T2 +15 · T1 stuck at 25.1 · T3 silent
redundancy	having more than you need, as a backup	3 temperature sensors instead of 1
TMR	Triple Modular Redundancy: 3 sensors and a vote	T1, T2, T3
vote / voter	choosing the value the majority agrees on	`voted T=25.0 C`
±1 °C tolerance (inexact voting)	values agree if they differ by at most 1 °C	24.9 and 25.1 agree
OK	all 3 agree	`[OK 3/3]`
MASKED	2 agree, the third is ignored – the fault is "hidden"	`T2=39.9 C out-voted`
DEGRADED	only 2 sensors left, but they agree – still works	`only 2 sensors`
FAILED	no agreement – the temperature cannot be trusted	`they DISAGREE`
WAITING	no vote yet (start or reset)	`WAITING -> OK`
out-voted	ignored because the others disagree with it	–
graceful degradation	working a bit less well instead of stopping	3 → 2 sensors
watchdog	a guard that checks every second that each sensor is still sending	`[WATCHDOG]`
heartbeat	every reading that arrives = "I am alive"	–
timeout	how long without a heartbeat before "dead"	1500 ms
RECOVERED	the sensor sends again	`T3 RECOVERED`

Word	Simple meaning	Example
validation	checking that a value is possible	115 °C → REJECTED
safe state / fail-safe	emergency mode that cannot hurt the plants: heater OFF, fan ON, alarm ON	`ENTER SAFE STATE`
good vote 1/2	the first trustworthy vote after a problem; 2 in a row are needed to leave the safe state	–
exception	a program error	`Exception in control cycle`
forward recovery	after an error: go to a known safe state and **continue**, without crashing	–
escalation / CRITICAL	safe state for more than 30 s → "a human must come"	`CRITICAL - manual intervention`

Control words

Word	Simple meaning	Example
controller	the part that decides what to switch	`[controller]`
control cycle	one round of decisions, every 1 s	–
Environmental Control	keep values inside limits by checking regularly	heater ON below 22 °C
Observe & React	react **immediately** when something dangerous is seen	alarm in 37 ms
hysteresis	two different limits for ON and OFF, so the device doesn't switch every second	heater ON < 22, OFF > 24

Concurrency (many threads) words

Word	Simple meaning	Example
producer / consumer	sensors **produce** readings, the consumer **uses** them	–
buffer / bounded buffer	a waiting box with a fixed size (10) between them	`buf 3/10`
monitor	a buffer that only one thread may use at a time, where threads wait politely (`wait`) and wake each other (`notifyAll`)	–
race condition	two threads change the same data at the same time and destroy each other's work	`LOST 69827 updates`
synchronized	"one at a time" lock	fix 1 in F4
AtomicInteger	a number that is changed in one unbreakable step	fix 2 in F4
volatile	a variable whose newest value all threads see at once	fault setting of a sensor
interrupt / clean shutdown	politely asking threads to stop, then waiting for them	`Consumer thread stopped (interrupted)`

Postman and program words

Word	Simple meaning	Example
Postman	a tool that sends requests to our program; here it plays "the outside world"	–

Word	Simple meaning	Example
request	one message from Postman	`POST /api/sensors/T2/fault`
GET / POST / DELETE	read data / send a change / remove something (= repair)	`DELETE .../T2/fault` = repair T2
REST API / endpoint	the program's "doors" for requests	`/api/status`
JSON	the data format of a request	`{"type":"WRONG_VALUE","value":15}`
400 / 404 / 405	wrong data / not found / wrong method	`REJECTED - 400`
scenario / folder	one test story in Postman	`F1-WrongValue`
[CHECK]	Postman reads the status to test the result; it changes nothing	–
SUMMARY	the list of everything important in one scenario	–
kernel	the part that starts and stops all jobs	`[KERNEL]`
log file	everything printed is also saved	`logs/2026-11-05/greenhouse_13-02-35.log`
events log	only the important lines	`logs/2026-11-05/events_13-02-35.log`

Quick tips

1. **Read the box, then the lines directly under it.** That is the reaction to that input.
2. **Ignore `[STATUS]` lines while reading a reaction.** They are only the heartbeat.
3. **Look for the marks:** `>>>` something happened, `!!!` danger, `+++` back to normal.
4. **Use slow mode for the presentation.** Choose the **Greenhouse-Presentation** environment in Postman.
5. **Read calmly afterwards.** Open `logs/<date>/events_<time>.log`. It has no status lines, only inputs, events and summaries.